

Some Mathematics behind `elbow-helper`

Warith Harchaoui

Summer 2026

Abstract

This note explains, from first principles, every piece of mathematics `elbow-helper` runs: the shipped single-knee pipeline (`robust_knee`, `robust_elbow`), the multi-knee research behind `robust_knees`, and the shared Gaussian-likelihood foundation every model-selection criterion in the package builds on. Citations live in `references.bib`; each claim states whether it is a verified citation, a citation reported through a secondary source, or this project’s own construction, following the evidence-marking convention used throughout this project.

How to read this note. Every concept gets the same treatment: first a plain-language explanation with a concrete example, assuming nothing beyond high-school algebra; then the precise formula, for a reader who wants to check every step, including a reader with a Ph.D. in applied mathematics who wants to verify the derivation rather than trust the prose. Skip the formula blocks on a first pass if the intuition already answers your question. Come back to them when it doesn’t.

An Introduction derives, once, the raw-likelihood-to-bounded-score machinery every later criterion reuses: why BIC, mBIC, and ICL all sit in natural log and how a deliberately pessimistic worst case, not an unbounded raw number, turns any of them into a justified, interpretable $[0,1]$ score. From there, the note is organized around a single question, asked three times with increasing realism: what does *one* knee mean geometrically (Part II), what does *several* knees mean geometrically (Part III), and how do you trust either answer once the data is noisy (Part IV)? Two of Part II’s worked examples extend onto real data rather than manufactured blobs or signal: the same k-means inertia curve and the same PCA scree plot, run for real on the 178 wines and thirteen real chemical measurements of the wine recognition dataset (Aeberhard et al., 1992), where `elbow_helper.robust_elbow` honestly abstains rather than confirm a knee that only outside knowledge of the data would make plausible.

Contents

I	Introduction	4
1	From raw likelihood to a bounded score	4
2	Related Work	6
II	One knee, the geometry of a bend	9
3	Putting the data on common ground: normalization	9
4	The difference-curve bump: what “knee” means, exactly	9
5	The broken-line model: a knee as a change of slope	10
6	A worked example: the k-means inertia elbow	10
III	Several knees, the geometry of many bends	12
7	The model: independent linear segments	12
8	Segment cost, in closed form	13
9	Optimal partitioning: the dynamic program	14
10	Greedy binary segmentation: the faster, worse alternative	15
11	A worked example: a mountain of alternating slopes	17
IV	One or several knees, with noise	18
12	Is there even a trend? Spearman rank correlation	18
13	Not trusting one scale: the smoothing $\times$ sensitivity search	19
14	Only count it if it survives everywhere: persistence clustering	19
15	Confirming the slope: the Theil-Sen contrast	19
16	Confirming the model: BIC and blocked cross-validation	20
17	Bootstrap: does the knee survive a redo?	22
18	The null test: could a straight line explain this by chance?	22
19	Plain BIC for several knees: the naive criterion and why it overselects	23

20	Modified BIC: Zhang and Siegmund's fix and a sign convention resolved by testing	24
21	ICL: from mixture models to changepoints, including a bug this project found and fixed	25
22	A sequential test controlling the family-wise error rate	26
23	Where this leaves the shipped API	27
24	A worked example: a sigmoid staircase	27
25	A worked example: the same subtle knee, at two noise levels	28
26	A worked example: the PCA scree plot	29
27	A worked example: an exponential saturation curve	30
28	A worked example: queueing latency and a capacity-planning knee	31
29	A worked example: cache sizing and diminishing returns	32
30	A worked example: when to stop testing	32

Part I

Introduction

A knee or an elbow, on a curve of diminishing returns, marks the point past which more input stops buying much more output. Existing detectors are good at proposing where that point sits; what they lack is a notion of confidence, so a single point estimate on a noisy curve is easy to over-trust in practice. `elbow-helper` wraps a from-scratch locator in a conservative decision procedure and refuses to answer outright when the evidence is weak.

This note derives, from first principles, every piece of mathematics the package runs. Several of its model-selection criteria, scores that reward a good fit while penalizing extra complexity, trace back to the same Gaussian log-likelihood: single-knee BIC, the Bayesian Information Criterion (Section 16); the several-knee $\text{BIC}(k)$ of Section 19; its stricter cousin the modified Bayesian Information Criterion, mBIC (Section 20); and a version that also charges for ambiguous segment boundaries, the Integrated Completed Likelihood, ICL (Section 21). This Introduction derives that shared foundation once, in natural log throughout, before any of those sections adds its own penalty term on top.

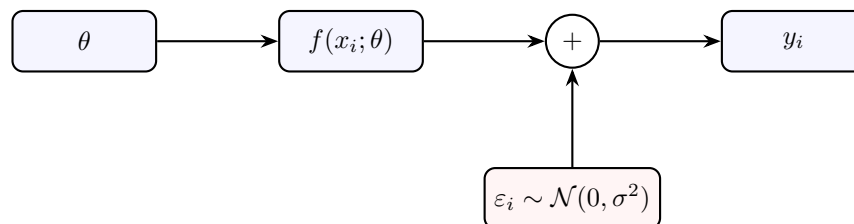
1 From raw likelihood to a bounded score

The notion, briefly. *Likelihood* usually names the joint density of the observed data, read as a function of the parameter rather than the data, $L_{\text{raw}}(\theta) := \prod_{i=1}^n p(y_i \mid x_i, \theta)$. This note instead takes likelihood *per observation* from the outset, an exponentiated *average* log-density rather than a raw product,

$$L(\theta, \sigma^2) := \exp(\mathbb{E}[\ln p]) = L_{\text{raw}}^{1/n},$$

the geometric mean of the n individual densities, exactly the n -th root of L_{raw} . That departure from the textbook definition, and why it is principled rather than a convenient rescaling, is the subject of a companion note, `LIKELIHOOD-en.tex`, factored out because that foundation is general and does not depend on curve-fitting at all: Jensen’s inequality and a temperature-indexed refinement borrowed from equilibrium statistical mechanics, Cramér’s large deviations and Wald’s martingale, a precise account of L as a statistical estimator with its bias and variance derived and verified by simulation, and how the same construction reads on a categorical classification model (with its own worked simulation) as well as the Gaussian regression model derived below. This note builds directly on top of it: every BIC-family criterion below inherits L ’s Gaussian instantiation. Section 22’s permutation-based sequential test is exactly the case `LIKELIHOOD-en.tex` identifies where Wald’s classical martingale bound does not directly apply, a composite rather than a fully specified alternative at each step.

Under a generative model $y_i = f(x_i; \theta) + \varepsilon_i$ with ε_i i.i.d. $\mathcal{N}(0, \sigma^2)$ (f any of the curve models this note fits, a single line, a broken line, or a k -segment piecewise line; θ its shape parameters):



The same handful of symbols recurs across every section below, often several sections apart, so a standing reference is worth more than holding it all in memory on a single pass.

Symbol	Meaning
n	number of points on the curve
x_i, y_i	the curve's own coordinates, point i
k	number of breakpoints (knees) in a fit
θ	a curve model's shape parameters
$f(x; \theta)$	the fitted curve (a line, a hinge, k segments)
σ^2	noise variance around the true curve
RSS	residual sum of squared errors, a fit's raw cost
MSE	RSS/ n , error averaged per point
NLL	negative log-likelihood, in nats
BIC, mBIC, ICL	NLL plus a complexity penalty
p	number of free parameters a model spends
S	sensitivity: how deep a dip counts as a real knee
α	the false-positive rate a sequential test controls
B	number of bootstrap or permutation replicates

Table 1: Notation glossary. Every symbol is reintroduced at its first use; this table only exists so nothing has to be memorized on a single pass.

$$L(\theta, \sigma^2) := \left(\prod_{i=1}^n (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{(y_i - f(x_i; \theta))^2}{2\sigma^2}\right) \right)^{1/n} \quad (1)$$

The product of n identical prefactors and the product of exponentials both collapse cleanly under the $1/n$ root: a product of exponentials is an exponential of a sum; that sum divided by n is a mean.

$$L(\theta, \sigma^2) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{1}{2\sigma^2} \cdot \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i; \theta))^2\right) = (2\pi\sigma^2)^{-1/2} \exp\left(-\frac{\text{MSE}(\theta)}{2\sigma^2}\right) \quad (2)$$

the average per-point squared error, the mean squared error $\text{MSE}(\theta) := \frac{1}{n} \sum_i (y_i - f(x_i; \theta))^2$. L is already an exp expression, base e , straight from the Gaussian density's own definition, before any analyst chooses to take a logarithm for convenience. Taking $\ln$ of L therefore does not introduce a base; it undoes the one already there:

$$\ell(\theta, \sigma^2) := \ln L(\theta, \sigma^2) = -\frac{1}{2} \ln(2\pi\sigma^2) - \frac{\text{MSE}(\theta)}{2\sigma^2} \quad (3)$$

Minimizing MSE over θ at fixed σ^2 is maximizing ℓ over θ , the reason ordinary least squares and Gaussian maximum likelihood are one procedure, not two. Profiling out σ^2 at its best-supported value, the maximum likelihood estimate (MLE) $\hat{\sigma}^2 = \text{MSE}$ and exponentiating -2ℓ , the conventional scaling for an information criterion, undoes the $\ln$ and lands on a single clean per-observation quantity, the measure information theory calls perplexity:

$$\text{PPL}(\theta) := \exp(-2\ell(\theta, \hat{\sigma}^2)) = 2\pi e \cdot \text{MSE}(\theta) \quad (4)$$

Scaling this per-observation perplexity back up to the full n -point sample recovers the negative log-likelihood, NLL, every BIC-family formula in this note is built from, written here through the

residual sum of squares, $RSS(\theta) := \sum_i (y_i - f(x_i; \theta))^2$, the un-averaged numerator MSE divides by n :

$$\text{NLL} := n \ln \text{PPL}(\theta) = n \ln \left(\frac{RSS}{n} \right) + n(1 + \ln 2\pi) \quad (5)$$

the same $n \ln(RSS/n)$ term every BIC-family formula in this note opens with.

Extreme cases. PPL is a genuine improvement over the raw NLL it is built from at exactly one end: NLL runs over $(-\infty, +\infty)$, but exponentiating automatically fixes the lower tail, $\text{PPL} = 2\pi e \cdot \text{MSE}(\theta) \geq 0$ always, with $\text{PPL} = 0$ at a perfect fit ($\text{MSE} \rightarrow 0$): the Gaussian degeneracy behind BIC’s whole reason to exist, a model with as many parameters as points drives MSE to zero and its raw likelihood to infinity. The upper end is still unresolved: $\text{MSE} \rightarrow \infty$ is not a meaningful worst case; it is merely unbounded, not a real reference point any fit would sit near. The sample mean $\bar{y}$ is not a good worst-case reference either: it is the *unique* constant that *minimizes* MSE among all constants, the best trivial predictor there is, not a bad one. That is exactly why normalizing against it (the textbook R^2) can still go negative: a real fit is easy to make worse than the one constant nothing else can beat. A genuinely pessimistic, finite worst case instead picks the single *observed* point that is hardest to predict everything else from:

$$y_{\text{bad}} := \arg \max_{y_j \in \{y_1, \dots, y_n\}} \frac{1}{n} \sum_i (y_i - y_j)^2, \quad \text{MSE}_{\text{worst}} := \frac{1}{n} \sum_i (y_i - y_{\text{bad}})^2 \quad (6)$$

Since $\bar{y}$ uniquely minimizes MSE among constants, any other constant, an actual observed value most of all, scores strictly worse: $\text{MSE}_{\text{worst}} > \text{Var}(y)$ always. Normalizing against this harder-to-beat ceiling instead,

$$1 - \frac{\text{MSE}(\theta)}{\text{MSE}_{\text{worst}}} = 1 - \frac{\text{PPL}(\theta)}{\text{PPL}_{\text{worst}}} \quad (7)$$

gives a score anchored at 1 for a perfect fit and at 0 for a fit exactly as bad as y_{bad} itself, with a far higher bar to cross into negative territory than the mean-based R^2 ever offers, since $\text{MSE}_{\text{worst}} > \text{Var}(y)$ by construction, built entirely from MSE against a real, deliberately pessimistic reference point, with no Bayes factor or posterior probability anywhere in the derivation. This is a stricter cousin of R^2 , not R^2 itself, R^2 keeps $\bar{y}$ as its baseline and accepts the occasional negative score in exchange. `elbow_helper.plotting`’s diagnostic legend takes a different route again, converting ΔBIC directly into a posterior probability via a Bayes factor (Kass and Raftery, 1995), a valid alternative normalization built on the same NLL but not the one derived here.

Every BIC-family quantity in this note inherits this same natural-log base by construction: each is NLL plus a complexity penalty, never a likelihood rederived from scratch. Changing the log base anywhere downstream would mean rederiving the Gaussian density itself in that base, not rescaling a finished number.

2 Related Work

Knee and elbow heuristics. Knee detection has one dominant reference point: Satopaa, Albrecht, Irwin and Raghavan’s Kneedle (Satopaa et al., 2011), built for exactly the setting this note opens with, a system operator facing a curve that needs an automatic answer to “where does this bend”. Kneedle’s mechanism is the same difference-from-chord geometry Section 4 builds: normalize the curve into the unit square, subtract the line joining its two endpoints, call the knee the point of maximum vertical distance from that chord. It is fast, lightly parameterized, and, on a clean curve,

usually right. What it does not do is say how right : Kneedle returns a single index whether the curve holds a genuine hand-built knee or three points of pure noise that happen to wiggle the right way, with no notion of how that answer would move under resampling, under a different smoothing scale, or against a null model with no knee at all. That gap, a point estimate with no confidence attached, is precisely what this whole note exists to close. `elbow-helper`’s locator keeps Kneedle’s geometric intuition at its core, the normalized difference-curve stays the origin of every candidate knee, but treats that candidate as a hypothesis to be tested (Sections 13–18) rather than an answer to hand back as is.

Changepoint detection and segmentation. Once the question moves from one knee to several, the relevant literature stops being knee detection and becomes changepoint detection. It splits along a cost-versus-flexibility line this note has to settle: an exact search that only earns its cost up to a bounded $k_{\max}$ or a greedy approximation that scales better but can lock in an early mistake. Killick, Fearnhead and Eckley’s pruned dynamic-programming algorithm, Pruned Exact Linear Time (PELT) (Killick et al., 2012), sits on the exact side, a penalized dynamic program pruned so that provably suboptimal candidate boundaries are discarded before they are even scored, recovering the same optimal segmentation an unpruned search would find at linear rather than quadratic cost, with k itself set by the penalty rather than fixed in advance. Section 9 rests on the same optimal-substructure argument, restricted here to a small, explicitly bounded $k_{\max}$ (few real curves plausibly hold more than a handful of genuine bends) rather than PELT’s unbounded, penalty-driven k , which is exactly what keeps the search small enough to run underneath this project’s bootstrap and permutation layers, something PELT itself has no equivalent of. Fryzlewicz’s Wild Binary Segmentation (Fryzlewicz, 2014) sits on the other side of that line: a recursive, greedy splitting procedure over randomly drawn subintervals, faster than exact search but liable to freeze in an early bad cut it can never reconsider, exactly the failure mode Section 10 demonstrates on a constructed example and the reason this project scores greedy segmentation as the faster, worse alternative rather than the default choice. Yao (Yao, 1988) was first to tie changepoint counting directly to Schwarz’s criterion, the paper the several-knee BIC(k) of Section 19 traces its ancestry to, even though, as that section shows, plain BIC’s assumptions do not survive contact with an unknown breakpoint location intact.

Model selection criteria. Every criterion this note builds answers the same question, is the added complexity worth it, with progressively less naive notions of what “complexity” costs. Schwarz (Schwarz, 1978) supplies the starting point: a Laplace approximation to a model’s marginal likelihood that charges a fixed $\frac{1}{2} \ln n$ per free parameter, correct for regular parametric families but silently assuming absent exactly the irregularity a breakpoint’s location introduces, an assumption Section 19 shows failing concretely rather than just in principle. Zhang and Siegmund (Zhang and Siegmund, 2007) diagnose that same irregularity and correct it with a segment-length-sensitive penalty, the mBIC of Section 20, though their own derivation leaves a genuine sign ambiguity this note resolves empirically rather than by assumption (again Section 20). Biernacki, Celeux and Govaert (Biernacki et al., 2000) approach complexity from an entirely different angle, originally built to pick the number of components in a mixture model, not a changepoint count at all, charging not just for parameters but for the resulting assignment ambiguity: two overlapping, hard-to- separate clusters cost more than two clean ones even at equal fit. Rigai, Lebarbier and Robin (Rigai et al., 2012) supply the machinery that makes carrying that idea over to changepoints practical, an exact posterior distribution via forward-backward passes over the whole discrete segmentation space, structurally the same recursion as a hidden Markov model’s forward-backward algorithm run over segmentations rather than hidden states, later accelerated by Cleynen, Luong, Rigai and Nuel (Cleynen et al., 2013). Section 21’s account of that adaptation includes a bug this project’s own tests caught directly: an early version that let the integrated-likelihood term dominate the entropy

correction and overestimate on pure noise, a failure mode the original mixture-model literature does not face in the same way. Kass and Raftery (Kass and Raftery, 1995) sit slightly apart from this whole family: not a complexity penalty but a decision-theoretic reading of a log-likelihood difference as an approximate Bayes factor, the sigmoid-based alternative normalization Section 1 derives and then sets aside in favor of the worst-case-anchored score used throughout the rest of this note.

Sequential and multiscale testing. A third family of methods answers “is this breakpoint real” directly, as a hypothesis test rather than a complexity-penalized score. Davies (Davies, 1977) identifies the central difficulty any such test has to solve: the breakpoint’s location is a nuisance parameter that exists only under the alternative, so the usual likelihood-ratio asymptotics do not apply, exactly the pivot problem Section 22’s sequential test sidesteps by using a permutation null instead of an asymptotic one. Frick, Munk and Sieling’s Simultaneous MUlti-scale Change-point Estimator (SMUCE) (Frick et al., 2014) solves a related but more ambitious problem, one confidence statement that holds across every smoothing scale at once, substantially more machinery than Section 22 needs for a sequential, one-breakpoint-at-a-time test whose validity is checked directly by simulation rather than by asymptotic theory. Both were considered and then set aside in favor of the permutation-based construction actually shipped, a deliberate trade: less theoretical generality for a guarantee this project could verify empirically rather than just cite.

Robust and classical statistics. The single-knee pipeline leans on older, narrower tools wherever an already well-understood robust classical method exists, rather than inventing a bespoke one. Spearman’s rank correlation (Spearman, 1904) screens for a monotonic trend before any knee search runs, chosen over a Pearson correlation precisely because it only asks whether y moves consistently with x , not whether that movement is linear, so a genuinely curved knee cannot fool it into concluding no trend is present. Theil (Theil, 1950) and Sen (Sen, 1968), developed twenty years apart and combined since into a single estimator, supply the slope estimate behind the Theil-Sen contrast: the median of every pairwise slope, a single extreme outlier among many ordinary points cannot move a median the way it would move an ordinary-least-squares fit. Efron’s bootstrap (Efron, 1979) is the resampling principle behind Section 16’s stability check, chosen over a closed-form asymptotic confidence interval precisely because this pipeline’s candidate knee has already survived several nonlinear, nonsmooth selection steps by the time a confidence statement is needed, exactly the setting where asymptotic normality is hardest to justify and resampling easiest.

Methodology. López de Prado (López de Prado, 2018) argues at length, from financial time series rather than curve fitting, for contiguous rather than shuffled cross-validation blocks whenever neighboring observations are correlated, since a shuffled split leaks a point’s own neighbors into its out-of-sample estimate. The same argument carries over directly to a curve’s x -axis: neighboring points on any smooth curve resemble each other for the same structural reason neighboring days of a price series do, which is why `elbow-helper`’s blocked cross-validation (Section 16) holds out contiguous blocks of x rather than randomly drawn points.

Further reading. Hastie, Tibshirani and Friedman (Hastie et al., 2009), Bishop (Bishop, 2006), Bouveyron, Celeux, Murphy and Raftery (Bouveyron et al., 2021), Cormen, Leiserson, Rivest and Stein (Cormen et al., 2022) and Dasgupta, Papadimitriou and Vazirani (Dasgupta et al., 2006) are cited throughout as accessible textbook treatments, not as the origin of any specific claim in this section; a few, drawn from the author’s own curated book list, are pointed to only where directly relevant.

Part II

One knee, the geometry of a bend

3 Putting the data on common ground: normalization

Intuition. A curve of website visits over months and a curve of stock prices over years look nothing alike numerically: one might range over thousands, the other over single digits, yet both could share the same *shape*. Before comparing shapes, both axes get rescaled into the same box, the unit square $[0, 1] \times [0, 1]$, so every threshold used later in the pipeline (how sharp a bend counts, how much noise is too much) means the same thing regardless of the data’s original units.

Formula. x is scaled linearly:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (8)$$

y is scaled the same way, but using the 5th and 95th percentiles instead of the raw minimum and maximum, then clipped to $[0, 1]$:

$$y_{\text{scaled}} = \text{clip}\left(\frac{y - y_{p5}}{y_{p95} - y_{p5}}, 0, 1\right) \quad (9)$$

The percentile choice is not cosmetic: a single wild outlier, a data-entry error, a one-off spike, would otherwise stretch the whole y axis to accommodate it, crushing the real signal into a sliver near zero. Percentiles absorb a handful of extreme points without distorting the shape that matters (`src/elbow_helper/preprocessing.py`).

4 The difference-curve bump: what “knee” means, exactly

Intuition. A textbook knee, shaped like $\sqrt{x}$, rises fast then levels off. Subtracting the straight diagonal line joining the curve’s two endpoints from the curve itself produces a bump: it starts at zero, peaks somewhere in the middle, and returns to zero. The peak of that bump sits exactly where the curve bends hardest away from a straight line, which is the intuitive definition of a “knee”. This picture turns into an algorithm (Satopaa et al., 2011): subtract the diagonal, find the bump’s peak, call it the knee. `elbow-helper`’s `locator.py` is a from-scratch NumPy port; see the project’s `README.md` Acknowledgements section for the implementation’s origin.

Four shapes exist in principle: concave-increasing, concave-decreasing, convex-increasing, convex-decreasing. Only one of them, concave-increasing, is the “obvious” bump-above-the-diagonal case. `elbow-helper` reorients the other three into that one case first (flip, mirror, or both, depending on `curve/direction`), runs the same bump-peak logic, then un-transforms the answer. This is why `curve` and `direction` matter: they tell the algorithm which of the four mirror images it is looking at.

Formula. After orienting to the concave-increasing frame, the difference curve is

$$d(x_i) = y_{\text{norm}}(x_i) - x_{\text{norm},i} \quad (10)$$

Its local maxima are candidate knees. At each local maximum j , a sensitivity threshold controls how far the curve must dip below the peak before that peak is trusted as a real knee rather than a wiggle:

$$T_j = d_{\max,j} - S \cdot |\overline{\Delta x_{\text{norm}}}| \quad (11)$$

where S is the sensitivity parameter: a larger S is more conservative, demanding a bigger dip. The bar denotes the mean spacing between consecutive x values. Traversing the difference curve left to right, a knee is declared at the last index before $d(x)$ drops below the currently active threshold T_j ; in “online” mode the traversal keeps re-arming at every later, higher peak, so only the last, most persistent bump wins (`src/elbow_helper/locator.py`, `find_knee`).

Worked example. Five normalized points, evenly spaced, $x_{\text{norm}} = (0, 0.25, 0.5, 0.75, 1)$, with $y_{\text{norm}} = (0, 0.6, 0.8, 0.9, 1)$, a $\sqrt{x}$ -shaped rise. The difference curve subtracts x_{norm} point by point: $d = (0, 0.35, 0.3, 0.15, 0)$, peaking at $d_{\max} = 0.35$ on the second point. The mean spacing $|\overline{\Delta x_{\text{norm}}}|$ is 0.25 here (four equal gaps covering a unit range). With sensitivity $S = 1$, the threshold sits at $T = 0.35 - 1 \times 0.25 = 0.10$. Walking the curve past the peak, $d = 0.3$ and $d = 0.15$ both still clear 0.10; only the last point, $d = 0$, drops below it. The knee is declared one step before that drop, at the fourth point, $x_{\text{norm}} = 0.75$: not the peak itself, but the last point still comfortably above the noise floor S sets. Raise S to 2 and the threshold rises to $0.35 - 2 \times 0.25 = -0.15$, below zero, so declaring is no longer picky about margin, the sole requirement is that $d(x)$ stay above the noise floor at all, an easy bar even a wiggle can clear.

5 The broken-line model: a knee as a change of slope

Intuition. A real knee should let a two-piece line (straight, then a different straight slope after the knee) fit the data noticeably better than a single straight line can. `elbow-helper` builds that two-piece model as one continuous curve using a hinge function, $\max(0, x - k)$, which is exactly zero before the knee k and grows linearly after it, so adding it to a plain line produces a single formula for a line that bends at k without jumping:

$$y = a + b x + c \cdot \max(0, x - k) \quad (12)$$

Worked example. Take $a = 1$, $b = 1$, $c = 2$, $k = 3$. Before the knee, $\max(0, x - 3) = 0$ for every $x \leq 3$, so the formula collapses to the plain line $y = 1 + x$: at $x = 2$, $y = 3$. Past the knee, the hinge term switches on: at $x = 5$, $\max(0, 5 - 3) = 2$, so $y = 1 + 5 + 2 \cdot 2 = 10$. Reading the slope off each side directly, before k it is $b = 1$; after k it is $b + c = 1 + 2 = 3$, three times steeper. Nothing jumps at $x = 3$ itself: both readings of the formula agree there, $y = 1 + 3 = 4$ from the left ($\max(0, 0) = 0$) and $y = 1 + 3 + 2 \cdot 0 = 4$ from the right, because the hinge term is continuous by construction, exactly zero at $x = k$ rather than switching on abruptly. c alone carries the whole story of the bend: $c = 0$ is just a plain line with no knee at all, and the further c sits from zero, in either direction, the sharper the change of slope at k .

This is the algebraic object every confirmation check in Part IV is ultimately built around: fitting it by ordinary least squares and comparing its residual error against a plain line’s answers the question “does a bend here actually help”.

6 A worked example: the k-means inertia elbow

`robust_elbow`’s own docstring names this the flagship convex-decreasing case. It is worth seeing end to end: run k-means for $k = 1, 2, \dots$, plot the inertia (within-cluster sum of squared distances) against k , and look for the point past which adding another cluster stops paying for itself.

`tests/test_real_world_examples.py::kmeans_inertia_curve` builds eight well-separated 2D blobs, each its own mildly elongated ellipse rather than a plain isotropic Gaussian and runs Lloyd’s algorithm (fifteen random restarts per k , to avoid a bad local optimum blurring the curve) for $k = 1, \dots, 24$. Inertia falls by two orders of magnitude at $k = 8$, the true cluster count. What happens after $k = 8$ is the point worth looking at closely: inertia keeps decaying, gently and a little unevenly, rather than snapping flat. That leftover slope is real, not noise in the plotting sense: each blob’s own elongation gives a k past 8 a little genuine sub-structure to exploit, exactly as a real dataset’s clusters are never perfect circles either. The figure below plots inertia on a logarithmic axis for this reason: the drop at $k = 8$ is so much larger than the tail’s own gentle decay that a linear axis would flatten the tail to the eye even though the underlying numbers keep falling.

Reading the evidence legend, ahead of its own derivation. The figure below is this note’s first with the full diagnostic legend `elbow_helper.plotting` draws next to every confirmed knee, five numbers Parts III–IV derive properly later; in plain language ahead of that: *detection probability* (Section 17) is how often the identical knee reappears under a residual bootstrap redo: close to 100% means the knee is not an artefact of this one particular noise draw. *null p* (null p -value, Section 18) is how surprising the observed knee would be if the true curve were a straight line: small means a straight-line explanation is implausible. *slope contrast* (Section 15) is a $[0, 1]$ score for how sharply the slope actually changes at the knee: near 100% means the two sides point in essentially opposite or wildly different directions. *posterior probability* (Section 16) is the BIC-derived odds, under a Bayes-factor approximation, that a broken line genuinely fits better than a single straight one rather than by chance. *fit quality* (Section 1, already derived) is how close the confirmed broken-line fit sits to the data relative to a deliberately pessimistic worst case: 100% at a perfect fit. Every one of the five is a bounded, justified $[0, 1]$ score, not a raw, scale-dependent statistic a reader has to calibrate by eye; four read strong evidence as a number near 100%, only null p runs the other way, small rather than large, exactly a p -value’s usual convention.

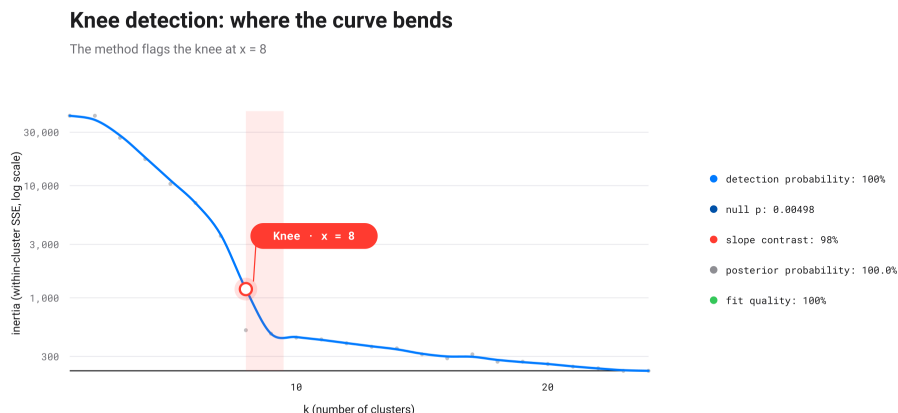


Figure 1: The k-means inertia curve on a log-scale y-axis, with `elbow_helper.robust_elbow`’s detected elbow (dashed red line) at $k=8$ (the true number of clusters) and a gently decaying tail past it.

Two practical traps this example surfaced, worth stating plainly. First: a k-means inertia curve has one point per candidate k , rarely more than a couple of dozen. Its genuine elbow usually sits close to the *left* edge, at a small k , not in the middle. `elbow-helper`’s defaults (Sections 13–18) are calibrated for longer, noisier, elbow-anywhere measurement curves:

`min_samples=20` and `min_side_points=5` reject a short, left-heavy curve like this one outright. `tests/test_real_world_examples.py` ships a `RobustKneeConfig` profile with both of those relaxed, plus several of the persistence and bootstrap thresholds loosened to match and states in a comment exactly why: short, clean applied curves are a genuinely different regime from the long, noisy time-series-like curves the defaults target, not a bug in either. Second: blocked cross-validation (Section 16) splits a curve this short, about 25 points, into `cv_folds` contiguous pieces. The default of 5 folds leaves so few points per fold that a fold boundary landing near the elbow can flip the sign of the held-out improvement on an unlucky seed. Three wider folds instead of five fixes it, a direct, checkable illustration of Section 16’s own point that cross-validation needs enough data per fold to say anything stable.

On real data, without manufactured blobs. The eight blobs above are clean by construction: hand-picked ellipses, well separated, with just enough irregularity to keep the tail from going flat. Real measurements are never that accommodating. The wine recognition dataset (Aeberhard et al., 1992), shipped with scikit-learn, gives 178 real Italian wines, each with thirteen real chemical measurements (alcohol, malic acid, ash, magnesium, total phenols, flavanoids, color intensity, among others) and a real cultivar out of three, never used for fitting, only for comparison after the fact. The thirteen measurements are standardized, then exactly the same k-means code as above, with the same relaxed short-curve `RobustKneeConfig` profile, runs for $k = 1, \dots, 12$ and calls `elbow_helper.robust_elbow` on the resulting inertia curve.

By eye, the curve bends around $k = 3$, which would line up nicely with the three real cultivars. `robust_elbow` still declines to confirm it: `NoClearKnee`, reason `NO_PERSISTENT_CLUSTER`. That is not a bug; it is Section 14 doing exactly what it exists to do. On real chemical measurements, the three cultivars’ profiles partly overlap, so the $k = 3$ candidate does not cluster together consistently enough across the smoothing widths and sensitivities tested to survive that check, even though a reader who already knows the answer would guess it correctly. This is exactly the design priority Section 18 stated on a synthetic curve: elbow-helper is built to fail in the honest direction, an abstention over a hasty confirmation, even when that means passing on an answer a human, armed with information the algorithm does not have, could have guessed right.

Part III

Several knees, the geometry of many bends

7 The model: independent linear segments

Intuition. Cut the curve into pieces at k breakpoints and fit each piece its own straight line, independently, allowing a visible jump at each cut rather than forcing the pieces to meet exactly. This is a deliberate departure from Part II’s continuous hinge model: once more than one breakpoint is free to move, a *continuous* piecewise-linear fit no longer decomposes into a sum of independent per-piece costs: moving one breakpoint changes the boundary condition every neighbouring piece must satisfy. That dependency breaks the efficient search algorithms below. The changepoint literature this section draws on and tests (Yao, Zhang and Siegmund, PELT, binary segmentation) is stated for independent segments, so matching that model is what makes the comparison a fair test of *their* claims, not a test of a different model wearing their name.

Wine, without manufactured blobs: the real inertia curve

178 real wines, 13 real chemical measurements, 3 real cultivars; robust_elbow called for real

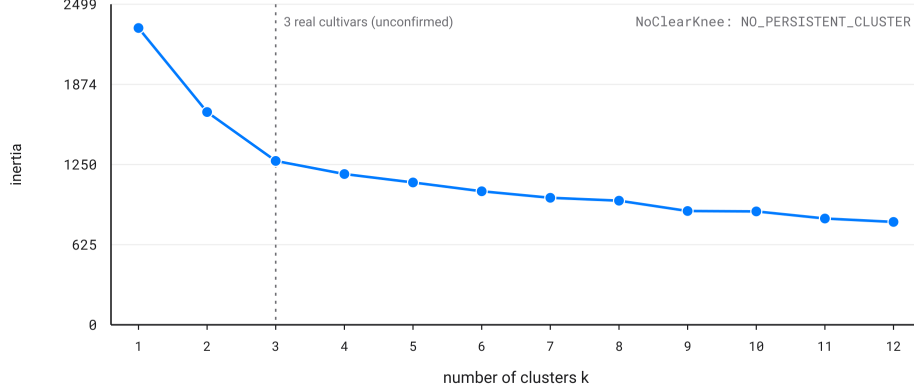


Figure 2: The real k-means inertia curve on the 178 wines and thirteen real chemical measurements of the wine recognition dataset (Aeberhard et al., 1992). The grey dotted line marks $k = 3$, the real number of cultivars, for comparison only: `elbow_helper.robust_elbow` draws no red line there, since it abstains (NoClearKnee, NO_PERSISTENT_CLUSTER) rather than confirm this candidate.

8 Segment cost, in closed form

Intuition. Fitting a straight line through a handful of points and measuring how well it fits, the residual sum of squares (RSS), sounds like it should be done point by point, but a shortcut exists: once five running totals are known (a sum of x 's, of y 's, of x^2 's, of xy 's, of y^2 's), the best-fit line's RSS for *any* stretch of points can be computed in one step, without touching the individual points again. Precomputing those totals once, then reading off any segment's cost instantly, is what makes searching over thousands of ways to cut the curve computationally realistic.

Formula. For a segment covering m points with values $x_1, \dots, x_m$ and $y_1, \dots, y_m$, let

$$S_1 = m, \quad S_x = \sum_i x_i, \quad S_y = \sum_i y_i, \quad S_{xx} = \sum_i x_i^2, \quad S_{xy} = \sum_i x_i y_i, \quad S_{yy} = \sum_i y_i^2 \quad (13)$$

The least-squares slope and intercept of $y = a + bx$ are

$$b = \frac{m S_{xy} - S_x S_y}{m S_{xx} - S_x^2}, \quad a = \frac{S_y - b S_x}{m} \quad (14)$$

and, substituting into $RSS = \sum_i (y_i - a - bx_i)^2$ and simplifying with the normal equations, the formula below is itself computed directly from the six sums, never touching the points again. The normal equations themselves, obtained by zeroing the two partial derivatives $\partial RSS / \partial a$ and $\partial RSS / \partial b$, read $S_y = ma + bS_x$ and $S_{xy} = aS_x + bS_{xx}$; these are exactly the two equations the formula for b and a above solves. Expanding the square in RSS gives $RSS = S_{yy} - 2aS_y - 2bS_{xy} + a^2m + 2abS_x + b^2S_{xx}$, where the group $a^2m + 2abS_x + b^2S_{xx}$ splits into $a(am + bS_x) + b(aS_x + bS_{xx})$; each parenthesis is exactly the right-hand side of a normal equation: $am + bS_x = S_y$ and $aS_x + bS_{xx} = S_{xy}$. That group therefore collapses to $aS_y + bS_{xy}$; putting it back into RSS cancels two of the four terms carrying a and b :

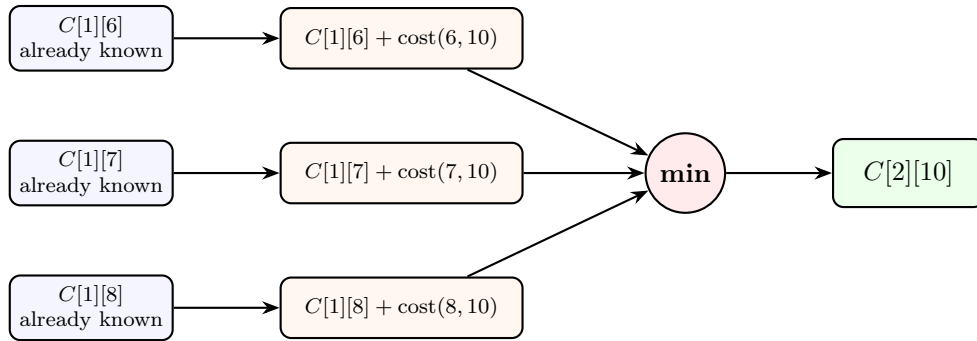
$$RSS = S_{yy} - 2aS_y - 2bS_{xy} + aS_y + bS_{xy} = S_{yy} - aS_y - bS_{xy} \quad (15)$$

Precomputing the six running sums once costs $O(n)$; every later segment-cost lookup then costs $O(1)$. This is standard practice in changepoint software, a direct algebraic consequence of the least-squares normal equations, not a result that needs its own citation.

9 Optimal partitioning: the dynamic program

Intuition. Choosing the single best way to place k breakpoints among n points by trying every combination would be astronomically slow. Dynamic programming avoids this by building the answer from smaller sub-answers: the cheapest way to explain the first t points with k segments is, for every possible location of the last cut s , the cheapest way to explain the first s points with $k - 1$ segments, plus the cost of one final segment from s to t , minimized over all valid s . Because the best k -segment answer depends only on the best $(k - 1)$ -segment answers, each already computed once, no combination is ever recomputed. This is Bellman’s principle of optimality.

Why the shortcut is actually valid. The claim above is not just a convenient assumption: it has a short proof by contradiction, the kind of argument worth seeing once rather than taking on faith. Suppose $C[k][t]$, the lowest total cost over every possible way to cut the first t points into $k + 1$ segments, is achieved by some particular partition whose very last cut sits at position s^* . That partition’s first k segments, covering points 1 through s^* , are themselves *some* k -segment partition of those points. Suppose, for contradiction, that they are not the *cheapest* such partition, that some other way of cutting the first s^* points into k segments costs strictly less. Swap that cheaper arrangement in for the first part, keep the same last segment from s^* to t unchanged: the result is still a valid $(k + 1)$ -segment partition of the first t points. Its total cost is strictly lower than $C[k][t]$, since only the part that got cheaper changed and nothing else did. That contradicts $C[k][t]$ being the minimum to begin with. So the first part of *any* optimal partition must itself be optimal for its own smaller sub-problem, exactly the property that lets the recursion below look up an already-solved answer, $C[k - 1][s]$, rather than re-solving that smaller problem from scratch at every step.



Computing $C[2][10]$ from three candidate last cuts, $s \in \{6, 7, 8\}$: each candidate reuses an already-solved $C[1][s]$, an answer computed once and looked up here rather than recomputed, adds the cost of one final segment, and the cheapest of the three candidates wins. A real run checks every valid s , not just three; the picture keeps a small slice for legibility.

Formula. With $C[k][t]$ the minimal total RSS of the first t points cut into $k + 1$ segments:

$$C[0][t] = \text{cost}(0, t) \quad (16)$$

$$C[k][t] = \min_s \left(C[k-1][s] + \text{cost}(s, t) \right) \quad (17)$$

where s ranges over cut points that leave every segment at least `min_seg` points long. Solving this for every k up to $k_{\max}$ and every t up to n costs $O(k_{\max} \cdot n^2)$, each cell an $O(n)$ minimization over $O(1)$ segment costs from the previous section. Backpointers recover the actual cut positions. `research/multiknee/tests/test_segmentation.py::test_dp_matches_brute_force_small_n` checks this against literal enumeration of every valid partition on small inputs, so its correctness is verified empirically, not just asserted from the textbook recursion (Cormen et al., 2022; Dasgupta et al., 2006).

This is the same recursion underlying Yao (1988), Zhang and Siegmund (2007), and PELT, “Pruned Exact Linear Time”, due to Killick, Fearnhead and Eckley (2012). PELT is this recursion with a pruning step added that provably never removes the optimum for an additive penalty, so it returns identical answers, only faster. No separate PELT implementation was built for that reason: it would reproduce these numbers exactly.

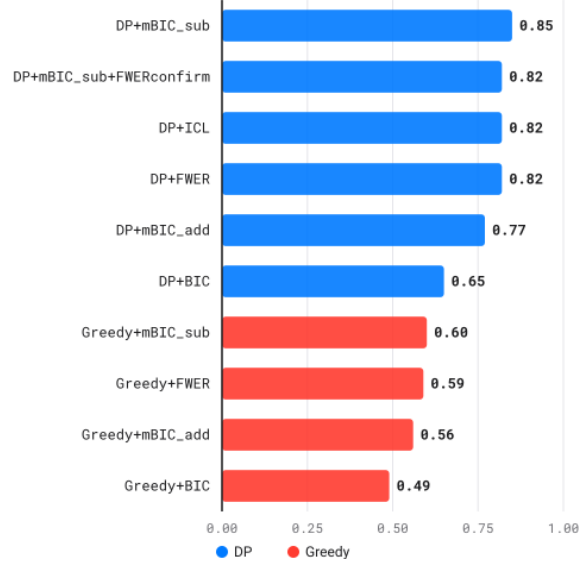
10 Greedy binary segmentation: the faster, worse alternative

Intuition. Instead of solving the whole problem exactly, repeatedly make the single best choice available right now: find the one split that most reduces the *global* error, commit to it, then repeat on the two resulting pieces. This is much faster than the dynamic program, but it can only ever produce worse or equal outcomes, never better, because it locks in early decisions before later evidence exists to correct them.

A concrete failure, not a hypothetical one. `research/multiknee/tests/test_segmentation.py::test_greedy_can_strictly[...]noiseless` exhibits a noiseless curve with two sharp, real breakpoints where greedy’s *first* split lands on neither of them; by the time greedy adds a second split, it still has strictly higher error than the dynamic program’s exact, zero-error optimum. `research/multiknee/compare.py` measures the downstream consequence directly, on synthetic piecewise-linear curves ($n = 100$, 25 replicates per true- k /noise cell): every DP-based method beats its greedy counterpart, DP+mBIC_sub reaching 0.85 exact- k accuracy against Greedy+mBIC_sub’s 0.60, DP+BIC reaching 0.65 against Greedy+BIC’s 0.49. Greedy’s early-commitment errors specifically cause it to *overselect* the number of breakpoints at low noise, since fixing greedy’s own placement mistake later looks, to a model-selection criterion, like genuine extra structure worth an extra breakpoint. This is the textbook motivation for Wild Binary Segmentation (Fryzlewicz, 2014).

Exact-k accuracy by method

$P(K = \text{true } k), 25 \text{ replicates} \times 4 \text{ true-}k \times 3 \text{ noise levels}$



False positives on flat data

$P(K > 0 \mid \text{true } k = 0)$

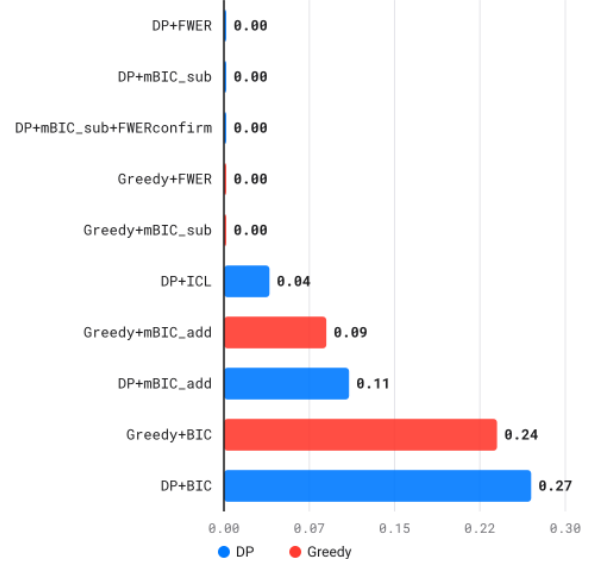


Figure 3: Left: exact- k accuracy, ranked, for every dynamic-program (blue) and greedy (red) combination with the four several-knee criteria this note derives, BIC, mBIC, ICL, and the FWER sequential test, measured on the same synthetic benchmark. Every DP-based method outranks its greedy counterpart. Right: the same ten methods' false-positive rate on genuinely flat data, plain BIC the clear outlier on both sides regardless of search strategy.

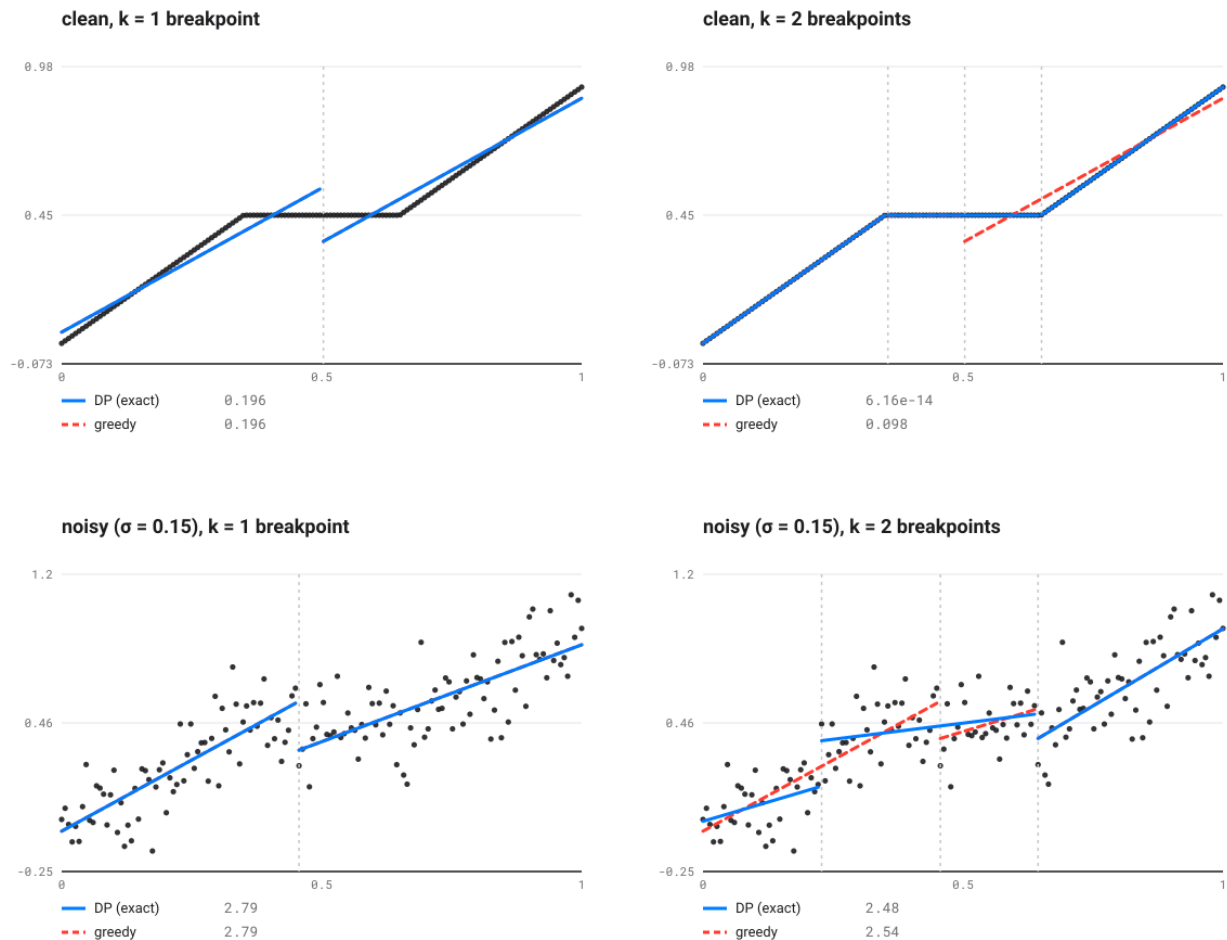


Figure 4: DP (solid blue) against greedy (dashed red) on the same curve, at $k = 1$ and $k = 2$ breakpoints, clean (top) and with $\sigma = 0.15$ noise added (bottom). At $k = 1$ the two coincide exactly, a single split has nothing to commit to early. At $k = 2$, clean, DP recovers the true breakpoints with essentially zero error while greedy’s first split (fixed at the $k = 1$ location) leaves it unable to reach the same fit at any later step, the concrete failure the text above describes. Noise softens the gap without closing it.

11 A worked example: a mountain of alternating slopes

Part II’s `robust_knee` needs `curve/direction` to know which of the four mirror images it is looking at or must infer it from the whole curve’s global shape. A curve that goes up, flattens, comes back down, and flattens again, a mountain, has no single global shape at all: Section 4’s fixed-sign geometry cannot describe it, no matter how `curve/direction` are set.

`robust_knees` sidesteps the question entirely, because this part’s independent-segment model never assumes a shared sign for the slopes in the first place: each segment gets its own free slope, so a sign change from one segment to the next is not a special case, just what the data happens to show.

`tests/test_multiknee.py::alternating_slope_curve` builds exactly this: steep up, flat, steep

down, flat, four segments and three breakpoints, each a genuine sign or magnitude change. `robust_knees` recovers all three, in the right order, with the right signs, on every seed tested.

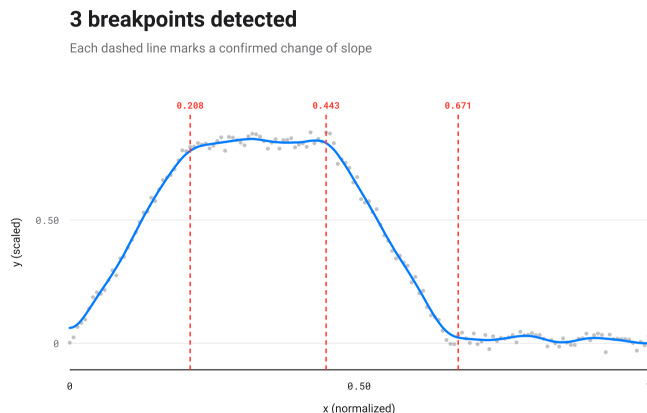


Figure 5: A mountain curve with `elbow_helper.robust_knees`’s three detected breakpoints (dashed red lines): up, flat, down, flat, recovered without any curve/direction argument.

Part IV

One or several knees, with noise

Parts II and III describe the geometry of a bend as if the data were exact. Real data never is, so nothing above is trusted on its own. This part is the trust layer: a chain of independent checks a single-knee candidate must survive to become a **ClearKnee**; a set of model-selection criteria that decide, for the multi-knee case, how many breakpoints the noise actually supports.

12 Is there even a trend? Spearman rank correlation

Intuition. Before hunting for a knee, `elbow-helper` checks a more basic question: does y move consistently with x at all? Not “is the relationship a straight line” (Pearson correlation asks that and would be fooled by a genuine, curved knee shape) but “when x goes up, does y tend to go up too or down, however wiggly the path”. The trick is to replace every value by its *rank*, 1st smallest, 2nd smallest, and so on, before correlating. Ranks discard exactly the information that would make an ordinary correlation sensitive to the curve’s exact shape or to outliers, keeping only the “does the order agree” question.

Worked example. Five points with $x = (1, 2, 3, 4, 5)$ and $y = (2, 1, 5, 3, 9)$. The y ranks are $(2, 1, 3, 4, 5)$ (rank 1 is the smallest value, so $y = 1$ at position 2 gets rank 1 and so on). Even though the raw values wiggle $(2, 1, 5, \dots)$, the ranks track x ’s ranks $(1, 2, 3, 4, 5)$ fairly closely, giving a high positive correlation despite the local dip.

Formula. With r_i, s_i the ranks of x_i, y_i (ties get the average rank of the tied group):

$$\rho = \frac{\sum_i (r_i - \bar{r})(s_i - \bar{s})}{\sqrt{\sum_i (r_i - \bar{r})^2} \sqrt{\sum_i (s_i - \bar{s})^2}} \quad (18)$$

`elbow_helper.numerics.spearman` implements this directly (no `scipy.stats.rankdata` dependency). A knee candidate is only screened in if $|\rho|$ clears `config.min_spearman_abs` (default 0.60) and a magnitude-weighted “how much of the movement fights the claimed direction” check also passes; see `INCOMPATIBLE_GLOBAL_SHAPE` in the README’s abstention-reason list ([Spearman, 1904](#)).

13 Not trusting one scale: the smoothing $\times$ sensitivity search

Intuition. Look at a noisy curve at native resolution and every little wiggle looks like a candidate knee. Smooth it heavily and a real knee can get blurred away entirely. Neither extreme is trustworthy alone, so `elbow-helper` runs the locator across a whole grid: several Gaussian smoothing widths, from 1, no smoothing, up to roughly a quarter of the data, crossed with several sensitivities S . Every run that finds a knee contributes one candidate to a pool; nothing is trusted yet, this stage only *proposes*.

14 Only count it if it survives everywhere: persistence clustering

Intuition. A genuine knee should show up at nearly every smoothing width and every sensitivity, wandering only slightly. A knee that is really noise tends to jump around unpredictably as the smoothing changes or shows up at only one or two settings and vanishes elsewhere. `elbow-helper` groups the candidate pool by location, within `cluster_tolerance` of each other, in normalized x units, then asks of the largest group: does it span several *consecutive* smoothing scales, does it show up across most sensitivities, is its spread (median absolute deviation) tight? If two groups are both large and comparably supported, the pipeline explicitly refuses to pick a winner (`MULTIPLE_PLAUSIBLE_KNEES`), rather than guess (`src/elbow_helper/clustering.py`).

15 Confirming the slope: the Theil-Sen contrast

Intuition. The ordinary way to estimate a slope, fit a line through a bunch of points, is easily thrown off by a single bad point far from the rest. The Theil-Sen estimator sidesteps this: compute the slope between *every pair* of points, then take the median of all those slopes. A handful of bad pairs, involving an outlier, get outvoted by the majority of good pairs.

Worked example. Three points $(0,0), (1,1), (2,100)$, the last one a wild outlier. Pairwise slopes: $(1-0)/(1-0) = 1$, $(100-0)/(2-0) = 50$, $(100-1)/(2-1) = 99$. The median of $\{1, 50, 99\}$ is 50, still pulled by the outlier with only three points, but with more normal points and one outlier, the median stops moving once outliers are a minority, unlike an ordinary least-squares fit, which the outlier would dominate immediately.

Formula.

$$\hat{\beta} = \text{median}_{i < j} \frac{y_j - y_i}{x_j - x_i} \quad (19)$$

`elbow-helper` computes this on a small window just left and just right of the candidate knee, then compares the two slopes:

$$\text{contrast} = \frac{|m_{\text{left}} - m_{\text{right}}|}{|m_{\text{left}}| + |m_{\text{right}}| + \epsilon} \quad (20)$$

A contrast below `config.min_slope_contrast` (default 0.30) fails this check (WEAK_SLOPE_CHANGE) (Theil, 1950) (Sen, 1968).

16 Confirming the model: BIC and blocked cross-validation

Intuition. Every extra free parameter in a model can only help it fit the *training* data better, purely mechanically, whether or not it captures anything real, a model with as many parameters as data points fits perfectly and explains nothing. The Bayesian Information Criterion, Schwarz’s fix for this, charges each extra parameter a fixed toll, in units of how much the log-likelihood must improve to be worth it, so that a model is only preferred if it clears that bar.

Where the log base comes from. Section 1 derives, from the raw Gaussian likelihood $L(\theta, \sigma^2)$, that the negative log-likelihood

$$\text{NLL} = n \ln \left(\frac{RSS}{n} \right) + n(1 + \ln 2\pi) \quad (21)$$

is natural-log by construction, inherited from the Gaussian density’s own exp, not chosen freely and that its extreme cases are best read through $\text{MSE}_{\text{worst}}$, a deliberately pessimistic finite baseline, rather than through $\text{MSE} \rightarrow \infty$ or the easily-beaten sample mean. Every BIC-family formula in this note, including the one below, opens with this same $n \ln(RSS/n)$ term.

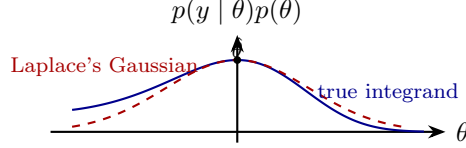
Where the $p \ln n$ penalty comes from. The complexity charge itself is not a rule of thumb; it falls out of asking a more honest question than “how well can this model fit if I pick its very best parameters”. A model’s *marginal likelihood* instead asks: averaged fairly over every parameter setting that seemed plausible before seeing the data, not just the one that happens to fit best, how well does this model explain what was actually observed? That average is an integral, $\int p(y | \theta) p(\theta) d\theta$, weighting each candidate θ by a prior belief $p(\theta)$ and by how well that particular θ explains the data. A model with many free parameters can reach an impressively high peak by tuning θ exactly to the data, but if only a thin sliver of parameter space near that peak actually fits well while the much larger space the integral has to average over fits badly, the honest average comes out lower than the peak alone would suggest: an automatic complexity tax a bare best-fit comparison never pays.

This integral rarely has a closed form, so it needs an approximation, and the *Laplace approximation* is the general recipe for integrating any sharply peaked function. Zoom in close enough on any smooth curve near its own peak and it looks like a parabola, the same idea a tangent line captures one derivative earlier, now one derivative further. Write $g(\theta) := \ln[p(y | \theta)p(\theta)]$ (the log of the very thing being integrated) and expand it to second order around its own peak $\hat{\theta}$ (where $g'(\hat{\theta}) = 0$):

$$g(\theta) \approx g(\hat{\theta}) - \frac{1}{2} |g''(\hat{\theta})| (\theta - \hat{\theta})^2.$$

Exponentiating turns that parabola into a Gaussian bump centered at $\hat{\theta}$. A Gaussian bump is exactly the one peaked shape whose area under the curve has a closed form already known from the standard Gaussian integral, $\int e^{-u^2/(2v)} du = \sqrt{2\pi v}$:

$$\int p(y | \theta) p(\theta) d\theta \approx e^{g(\hat{\theta})} \sqrt{\frac{2\pi}{|g''(\hat{\theta})|}}.$$



The true integrand (solid) need not be symmetric at all; the Laplace approximation (dashed) is the Gaussian bump that matches its height and its curvature at the single peak $\hat{\theta}$. The two nearly coincide right at the peak, which is exactly the region carrying almost all of the integral’s area and drift apart in the tails, which barely contribute area either way.

Taking $\ln$ of the approximation above turns the product into a sum, $\ln[\text{marginal likelihood}] \approx g(\hat{\theta}) + \frac{1}{2} \ln(2\pi) - \frac{1}{2} \ln |g''(\hat{\theta})|$. The last piece is where n and p enter: fitting p parameters to n independent observations, the curvature $|g''(\hat{\theta})|$ grows in proportion to n in each of the p parameter directions, so $\ln |g''(\hat{\theta})|$ grows like $p \ln n$ plus pieces that do not grow with n at all. As n grows, that one $p \ln n$ term dominates every other piece in the expansion, the prior $\ln p(\hat{\theta})$ included, so those smaller pieces are dropped and only the part that actually keeps growing survives: $-\frac{1}{2} p \ln n$. Multiplying through by -2 , the same $-2 \ln(\cdot)$ scaling Section 1 already used to turn a log-likelihood into an information-criterion-style number, turns that into exactly $+p \ln n$: Schwarz’s penalty, not asserted but read off the leading term of an honest average over parameter space.

Formula. Schwarz’s fix for the overfitting problem in this section’s Intuition paragraph adds that complexity penalty, $p \ln n$, on top of this same NLL. Dropping the additive constant $n(1 + \ln 2\pi)$, which cancels when comparing two models on the same data:

$$\text{BIC} = n \ln \left(\frac{RSS}{n} \right) + p \ln n \quad (22)$$

Lower is better. `elbow_helper.numerics.bic` implements exactly this; the single-knee pipeline compares the BIC of the single line ($p = 2 + 1$) against the broken line ($p = 3 + 1$) from Section 5, requiring an improvement of at least `config.min_bic_improvement` (default 10) (Schwarz, 1978) (Hastie et al., 2009) (Bishop, 2006).

From ΔBIC to a bounded score. A raw $\Delta\text{BIC} = \text{BIC}_{\text{single}} - \text{BIC}_{\text{broken}}$ has no natural ceiling: “10” and “282” both favour the broken line, but nothing about the number itself says how confident that is; the scale shifts with sample size and noise too. The diagnostic figure (`elbow_helper.plotting`) instead reports a probability, a score that must land in $[0, 1]$, with 0 and 1 as the two extreme cases: total confidence in the single line or in the broken one. That requirement is what fixes the log base, not a free choice: Kass and Raftery (1995) (eq. 4) show $\Delta\text{BIC} \approx 2 \ln(\text{BF})$, where BF is the Bayes factor between the two models; the $\ln$ on the right is the *same* natural log already used to build BIC above, inherited from its $-2 \ln(\hat{L})$ definition. Undoing that $\ln$ therefore needs the matching inverse, $\exp$ base e , not an arbitrary rescaling by $\ln 10$ or $\ln 2$, which would only relabel an unbounded number, not bound it:

$$\text{BF} \approx \exp \left(\frac{\Delta\text{BIC}}{2} \right) \quad (23)$$

Under equal priors, posterior odds equal BF, so the posterior probability of the broken-line model is $\text{BF}/(1 + \text{BF})$, which is exactly the logistic sigmoid of $\Delta\text{BIC}/2$:

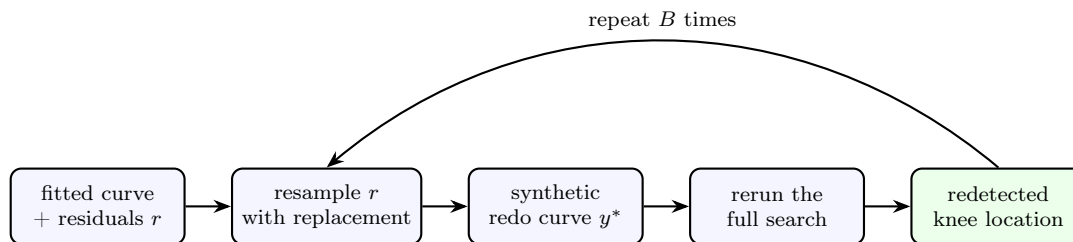
$$P(\text{broken line} \mid \text{data}) = \frac{1}{1 + \exp(-\Delta\text{BIC}/2)} \quad (24)$$

bounded in $[0, 1]$ by construction, 0.5 at $\Delta\text{BIC} = 0$ (no evidence either way), and saturating smoothly toward the 0/1 extremes as the evidence grows, in either direction, rather than growing without limit the way the raw ΔBIC does.

BIC only checks whether the extra parameter is worth its penalty *on the data used to fit it*. Blocked cross-validation asks a complementary, more paranoid question: does the broken line still win when tested on data it never saw while fitting? Ordinary shuffled cross-validation would leak information here, since neighbouring points on a curve are similar, so **elbow-helper** holds out *contiguous* chunks of x in turn, not random points, mimicking how the curve would look with a genuine, unseen chunk missing. This is the same validation-rigor concern Marcos López de Prado’s book on financial machine learning argues for at length, in the context of ordered, sequential data (2018).

17 Bootstrap: does the knee survive a redo?

Intuition. If the data collection were repeated, would the same knee show up again? Or was this run’s knee a fluke of this run’s particular noise? Since a real redo is not available, the bootstrap simulates one: take the residuals left over after fitting the accepted broken-line model, the part of y the model did not explain, resample them with replacement, add the resampled residuals back onto the fitted model to build a synthetic “redo” curve, then rerun the *entire* search on it. Repeat many times, Efron’s bootstrap. A knee that shows up only in the original run and rarely in the resampled redos was probably a fluke.

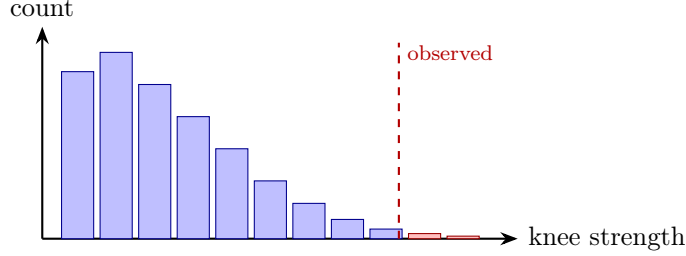


One bootstrap replicate, left to right. The loop on top runs the whole chain B times, so what comes out is not one redetected location but a whole distribution of them, the raw material for both the detection rate and the 90% interval below.

Formula. For each of B replicates, $y^* = \hat{y} + r^*$ where r^* is a resample-with-replacement of the observed residuals. **elbow-helper** requires the knee to be redetected in at least `config.min_bootstrap_detection_rate` (default 90%) of replicates, with a tight, unimodal 90% interval, the 5th-95th percentile spread of the redetected locations (Efron, 1979) (Hastie et al., 2009).

18 The null test: could a straight line explain this by chance?

Intuition. The last, most skeptical check: simulate many curves that have *no* real knee at all, straight lines carrying noise at the same estimated scale as the accepted model’s residuals, run the exact same search-and-confirm procedure on each, and count how often that procedure still manages to report a knee as strong as the one actually observed. If pure noise regularly produces something this strong, the observed knee is not trustworthy evidence of a real bend, it is just what noisy straight lines look like some of the time.



The null distribution of knee strength across B simulated straight-line-plus-noise curves (blue), with the observed knee’s own strength marked (red dashed line). p is the fraction of that null distribution sitting at or beyond the mark (the shaded red bars): a small p means pure noise essentially never reaches what was actually observed.

Formula. With $B = \text{config.null_replicates}$ (default 200) Monte Carlo replicates and a lexicographic, search-adjusted test statistic, so a null replicate only “beats” the observed knee if it passes *the same* confirmation gates, not just a raw score:

$$p = \frac{1 + \#\{\text{null replicates at least as strong as observed}\}}{B + 1} \quad (25)$$

The $+1$ in numerator and denominator is the standard finite-sample correction that keeps a Monte Carlo p -value from ever claiming exactly zero. `elbow-helper` requires $p \leq \text{config.max_null_p_value}$ (default 0.01) (`src/elbow_helper/null_test.py`, whose own module docstring states this exact formula).

Only a candidate that clears every check in the preceding sections becomes a `ClearKnee`.

19 Plain BIC for several knees: the naive criterion and why it overselects

Intuition. Section 16’s BIC penalizes the number of free *regression* parameters, but choosing *where* to place k breakpoints among roughly n possible positions is itself a form of freedom, one that plain BIC never charges for. It is as if a multiple-choice exam only docked points for wrong answers on the questions attempted, ignoring how many questions there were to choose from in the first place.

Formula. With $p = 2(k + 1) + 1$, two regression parameters per independent segment, plus the noise variance:

$$\text{BIC}(k) = n \ln \left(\frac{\text{RSS}(k)}{n} \right) + p \ln n \quad (26)$$

The consequence is not just theoretical: Figure 3 measures a 27% false-positive rate for this exact criterion (DP+BIC) on data with no real breakpoints and only 0.65 overall exact- k accuracy, the worst of the four DP-based criteria tested (Yao, 1988) (Zhang and Siegmund, 2007).

The deeper reason plain BIC undercharges is that a breakpoint’s location is not a “regular” parameter in the technical sense BIC’s derivation assumes. The log-likelihood is not smooth, twice differentiable, in the location, so the location estimator does not converge at the usual $\sqrt{n}$ rate with a Gaussian limit; it converges faster, at rate n , with a limiting distribution given by the argmax of a random walk. Charging $\frac{1}{2} \ln n$ per parameter, the Laplace-approximation argument behind BIC, assumes exactly the regularity that breaks down here.

20 Modified BIC: Zhang and Siegmund’s fix and a sign convention resolved by testing

Intuition. If plain BIC undercharges for the freedom to place breakpoints, the fix is to charge more, specifically to charge *more for very short or very uneven segments*, since a tiny segment offers far more placement freedom, many nearby positions all look almost as good, than a segment that spans a large, well-defined chunk of the data.

Formula. Zhang and Siegmund’s penalty, reported here through a secondary source rather than the primary paper, is

$$\text{penalty}(k) = 3k \ln n + \sum_{j=1}^{k+1} \ln\left(\frac{\ell_j}{n}\right) \quad (27)$$

where ℓ_j are the resulting segment lengths. The 3, instead of plain BIC’s implicit 2 from the two regression parameters alone, is already a stronger complexity charge on its own; the second term is always ≤ 0 and, by the concavity of $\ln$, sits closest to zero when segments are balanced and most negative when they are very uneven.

Worked example. Take $n = 100$ and $k = 1$, one breakpoint, two segments. Split evenly, $\ell = (50, 50)$: the second term is $\ln(0.5) + \ln(0.5) \approx -1.386$. Split unevenly, $\ell = (10, 90)$: it is $\ln(0.1) + \ln(0.9) \approx -2.303 - 0.105 = -2.408$, well past the balanced case, exactly the concavity of $\ln$ making a small segment cost disproportionately more than a large one gains back. Both cases share the same $3k \ln n = 3 \ln 100 \approx 13.816$. Whether that leaves the uneven split more or less penalized than the even one depends entirely on the sign chosen next.

A genuine ambiguity, resolved empirically rather than assumed. Folding this penalty into a lower-is-better criterion by direct addition,

$$\text{mBIC}_{\text{additive}}(k) = n \ln\left(\frac{RSS(k)}{n}\right) + 3k \ln n + \sum_{j=1}^{k+1} \ln\left(\frac{\ell_j}{n}\right) \quad (28)$$

makes the most-negative, uneven-segment case *lower* the total, so this combination *rewards* uneven segments, the opposite of the “penalizes short and uneven segments” behaviour the term is meant to have. Plugging the worked numbers back in: additive contributes $13.816 + (-1.386) = 12.430$ for the balanced split but only $13.816 + (-2.408) = 11.408$ for the uneven one, a *lower* number on a lower-is-better criterion, rewarding exactly the split that should cost more. Subtracting the same term instead,

$$\text{mBIC}_{\text{subtractive}}(k) = n \ln\left(\frac{RSS(k)}{n}\right) + 3k \ln n - \sum_{j=1}^{k+1} \ln\left(\frac{\ell_j}{n}\right) \quad (29)$$

does penalize uneven segments, matching the literature’s stated intent: the same two splits now contribute $13.816 - (-1.386) = 15.202$ balanced against $13.816 - (-2.408) = 16.224$ uneven, the uneven split correctly costing more. Both forms are implemented and directly compared: `modified_bic_subtractive` reaches 0.85 overall accuracy and a 0% false-positive rate at true $k = 0$; `modified_bic_additive` still beats plain BIC (0.77 vs. 0.65) but leaves an 11% false-positive rate. **The subtractive form is the one shipped.**

21 ICL: from mixture models to changepoints, including a bug this project found and fixed

Intuition. Biernacki, Celeux and Govaert’s Integrated Completed Likelihood, developed for choosing the number of clusters in a mixture model, adds one more consideration on top of BIC: not just “does this model fit well relative to its complexity”, but “are the resulting groups actually unambiguous”. Two overlapping, hard-to-tell-apart clusters are penalized even if they fit the data about as well as two clean, well-separated ones. Gilles Celeux, a co-author of the original ICL paper, later co-wrote a book-length treatment of exactly this family of methods (Bouveyron et al., 2021).

For changepoints, the analogue of “which cluster does this point belong to” is “where exactly is the cut”: the relevant uncertainty is over the *entire discrete segmentation*, not over individual point-to-component labels. Rigaiil, Lebarbier and Robin supply the machinery: an exact, non-asymptotic posterior over every way to place $K - 1$ breakpoints, via a forward-backward dynamic program structurally identical to the forward-backward algorithm of a Hidden Markov Model, run over segmentations instead of hidden states (2012); Cleynen, Luong, Rigaiil and Nuel build directly on this (2013).

Formula. A forward pass computes, in log space, the total likelihood mass over every valid K -segment partition of the first t points, using the Gaussian segment log-likelihood

$$\ell(i, j) = -\frac{RSS(i, j)}{2\sigma^2} - \frac{j - i}{2} \ln(2\pi\sigma^2) \quad (30)$$

$$\log Z_1(t) = \ell(0, t), \quad \log Z_K(t) = \log \sum_s \exp\left(\log Z_{K-1}(s) + \ell(s, t)\right) \quad (31)$$

The entropy of the resulting discrete posterior over segmentations, $H(K) = \log Z_K(n) - \mathbb{E}[\ell(S)]$, is estimated by Monte Carlo backward-sampling, drawing complete segmentations from the posterior and averaging their log-probabilities, rather than derived in closed form.

A bug the tests caught, not a design choice. The first version built here defined the score directly as $-\log Z_K(n) + H(K)$, pure integrated likelihood plus entropy, no separate complexity penalty, reasoning that the exact posterior already “contains” whatever complexity notion is needed. Testing on sixty points of pure noise falsified that: $\log Z_K(n)$ rose from 89.8 ($K = 1$) to 94.7, 99.0, and 102.8 nats as K grew to 4, purely from summing over combinatorially more candidate segmentations, while entropy only rose from 0 to 3.7, 5.9, and 7.9 nats over the same steps, not fast enough to cancel the growth. The resulting score kept improving as K grew even on flat noise, exactly the overselection failure this whole exercise exists to avoid.

The fix follows from re-reading the mixture-model form above: ICL is *BIC plus* an entropy correction, not a replacement for BIC’s own complexity penalty:

$$\text{ICL}(k) = \text{BIC}(k) + 2H(K), \quad K = k + 1 \quad (32)$$

Retested on the same flat-noise data, this version correctly selects $k = 0$; Figure 3 shows it performing in the same tier as the subtractive mBIC (0.82 vs. 0.85 overall accuracy) once corrected (Biernacki et al., 2000).

One log base, top to bottom. The Gaussian segment log-likelihood $\ell(i, j)$ above, plain $\text{BIC}(k)$ (Section 19), $\text{mBIC}_{\text{subtractive}}(k)$, and $\text{ICL}(k) = \text{BIC}(k) + 2H(K)$ are all written in *natural* log throughout this project’s implementation, `numerics.py`’s single-knee BIC, `multi_criteria.py`’s mBIC, and `research/multiknee/criteria.py`’s $\ell / \log Z_K / \text{ICL}$ all call `np.log`, never `np.log10` or `np.log2`. That is not an implementation detail to normalize away: it is exactly what makes Section 16’s $\Delta\text{BIC} \approx 2\ln(\text{BF})$ argument and the exp-then-sigmoid conversion into a $[0, 1]$ -bounded

posterior probability built on it apply unchanged to a difference between any two of these four quantities, not only to plain single-knee BIC. ΔmBIC or ΔICL between two candidate segmentations is, by the same substitution, $2\ln$ of an approximate Bayes factor between them, so the identical base- e $\exp(\cdot/2)$ and logistic-sigmoid steps turn it into a bounded confidence score, with 0 and 1 the same two extreme cases as before. `elbow-helper` does not ship this conversion for the multi-knee criteria (the shipped pipeline gates k via the FWER test of the next section instead), but the derivation carries over exactly, because the log base was never a free choice to begin with.

22 A sequential test controlling the family-wise error rate

Intuition. Independent of any BIC-family score, a more direct question can be asked at each step: walking the sequence of segmentations $k = 1, 2, \dots$, is the error reduction from adding breakpoint k bigger than pure chance would produce? This is answered the way a permutation test answers any such question: resample what “chance” looks like many times and see how often chance alone beats what was actually observed.

Formula. At each step, residuals $r = y - \hat{y}_{k-1}$ under the accepted $(k-1)$ -breakpoint model are resampled with replacement, Efron’s bootstrap again, to build B synthetic curves $y^* = \hat{y}_{k-1} + r^*$. On each, the largest possible error reduction from one more split anywhere is computed, giving a null distribution:

$$p = \frac{1 + \#\{\text{replicates with null reduction} \geq \text{observed reduction}\}}{B + 1} \quad (33)$$

Breakpoint k is accepted only if $p \leq \alpha/k_{\max}$, a Bonferroni correction across up to $k_{\max}$ sequential tests. The procedure stops at the first breakpoint that fails, so it can never “skip” a rejected breakpoint and accept a later one. Bonferroni is conservative for independent tests; these sequential tests are not independent, each conditions on the model accepted at the previous step, which keeps the true family-wise error rate (FWER), the probability of accepting at least one spurious breakpoint anywhere in the sequence, at most as large as the nominal α , not larger.

Figure 3 shows this reaching a 0% false-positive rate at true $k = 0$ and 0.82 overall accuracy, in the same tier as the subtractive mBIC and the corrected ICL. Two more rigorous alternatives were considered and not implemented: Davies’ asymptotic test (1977) and the multiscale guarantee of SMUCE, “Simultaneous Multi-scale Change-point Estimator” (Frick et al., 2014), in favour of a permutation test whose correctness is checkable directly by simulation.

The four criteria, side by side. Sections 19 through 22 build up four different answers to the same question, how many breakpoints does the data actually support, each fixing a gap the previous one left open. Table 2 collects them in one place, with every number pulled directly from the measurements already reported above rather than restated from memory.

Criterion	Penalizes	Blind spot it fixes	FP @ true $k=0$	Overall ac- curacy
Plain BIC(k)	regression parameters only, $p \ln n$	nothing: treats a breakpoint's location as a regular parameter it is not	27%	0.65
mBIC _{sub}	parameters plus uneven segment lengths, $3k \ln n - \sum \ln(\ell_j/n)$	plain BIC's blindness to placement freedom	0%	0.85
ICL	BIC(k) plus segmentation ambiguity, $+2H(K)$	ambiguous segment boundaries a fit-only score cannot see	0% (corrected)	0.82
FWER sequential test	nothing additive: a permutation p -value per breakpoint, Bonferroni-corrected	every BIC-family blind spot at once, by testing directly instead of penalizing	0%	0.82

Table 2: The four several-knee criteria this note derives, compared on the same synthetic benchmark (Figure 3). Lower is not always better across rows, each formula has its own scale, only within a row's own comparisons; the two right-hand columns are the common yardstick.

23 Where this leaves the shipped API

`elbow_helper.robust_knees` ships exactly the combination validated above: dynamic-program search (Section 9), the subtractive-sign modified BIC as the primary criterion (Section 20), and the FWER sequential gate (Section 22) layered on top by default, taking $\min(\text{mBIC's } k, \text{FWER's } k)$, matching `elbow-helper`'s stated design priority of minimizing false-positive knees even at the cost of more abstentions. Plain BIC, the additive mBIC, ICL, and greedy search are kept in `research/multiknee/` as tested-and-rejected, or tested-and-costly, alternatives that justify that choice, not shipped as public configuration options, so the public API surface reflects what was actually shown to work.

24 A worked example: a sigmoid staircase

Every section above is either a formula or a claim checked by a test. Here is one curve, run through the actual shipped code, to see the whole chain land somewhere concrete.

A logistic sigmoid, $1/(1 + e^{-\text{steepness}(x-\text{center})})$, rises smoothly from 0 to 1 around its center and has no literal breakpoint anywhere: it is infinitely differentiable, with no kink for a piecewise-linear search to latch onto in principle. Summing three sigmoids at different centers, each scaled to add one “step”, builds a smooth staircase: three rises separated by flat plateaus, the kind of shape that shows up whenever several independent thresholds get crossed in sequence, three separate user cohorts each ramping up adoption in a different week, three separate sensors each saturating at a different load.

A piecewise-*linear* model, run against a smooth curve like this, cannot represent a rise with a single straight segment without either cutting a flat plateau short or cutting the rise short. The compromise `robust_knees` settles on is visible directly in the figure below: each of the three rises gets bracketed by *two* breakpoints, one where the flat plateau ends and the rise begins, one where the rise ends and the next plateau begins. Three rises, six breakpoints, no exceptions across five different noise draws tested (`tests/test_multiknee.py::test_sigmoid_staircase_brackets[...]breakpoint_pair`).

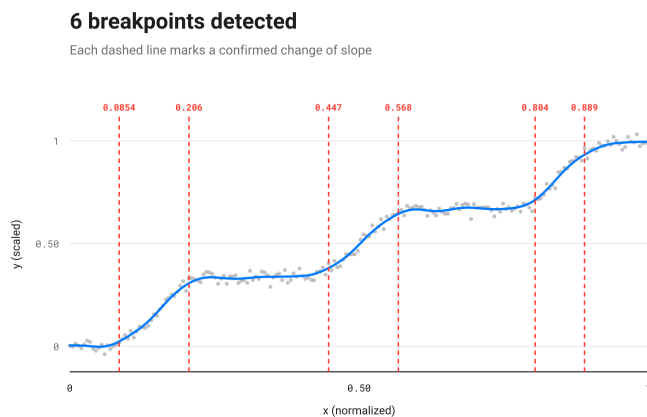


Figure 6: A smooth staircase built from three logistic sigmoids, with `elbow_helper.robust_knees`’s six detected breakpoints (dashed red lines) each bracketing one rise’s start and end.

One practical trap this example surfaced, worth stating plainly: the FWER gate’s Bonferroni correction divides α by $k_{\max}$, so the smallest p -value the permutation test can produce, $1/(n_{\text{permutations}} + 1)$, must stay below $\alpha/k_{\max}$. Otherwise the gate can never pass no matter how real the effect is. With the default $k_{\max} = 4$ and `fwer_permutations` = 200, $1/201 \approx 0.005 < 0.05/4 = 0.0125$, comfortably clear. Push $k_{\max}$ up, as this six-breakpoint example needs, without also raising `fwer_permutations`, silently locks the gate out: this is exactly what happened on the first attempt at this example ($k_{\max} = 8$, the default 100 permutations, every single breakpoint rejected with $k = 0$), caught by looking at the diagnostics rather than assuming the algorithm had failed for some more mysterious reason.

25 A worked example: the same subtle knee, at two noise levels

Every check in Sections 13–18 exists to answer one question honestly: is this knee real? Or is it what noise looks like sometimes? The cleanest way to see the answer is to hold the true shape fixed and only change the noise.

`tests/test_multiknee.py::subtle_knee_curve` is deliberately unspectacular: a small, genuine slope change at $x = 0.5$, nowhere near as dramatic as the sigmoid staircase or the mountain. At low noise, `robust_knees` detects it reliably, on every seed tested. At high noise, with the identical true shape underneath, it mostly abstains, reporting zero breakpoints rather than a guess.

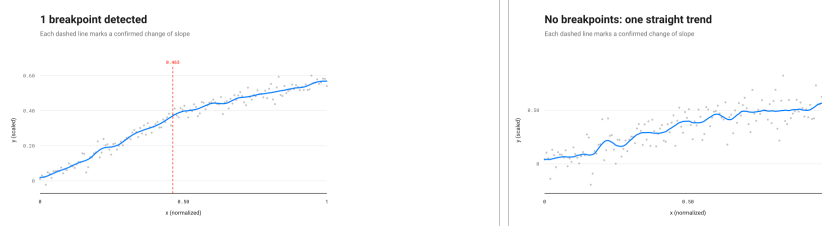


Figure 7: The same subtle knee at two noise levels: low noise (left) is detected with one breakpoint, high noise (right) abstains with zero, the same true shape underneath both panels.

This is the design priority from Section 18 made visible on one curve: `elbow-helper` is built to be wrong in the honest direction. A missed real knee at high noise is a cost this library accepts deliberately; a reported knee that is really noise is the failure mode every check in Part IV exists to rule out.

26 A worked example: the PCA scree plot

The other case `robust_elbow`’s docstring names directly: a scree plot, the eigenvalues of a covariance matrix in decreasing order, used to decide how many principal components carry real signal versus noise.

`tests/test_real_world_examples.py::pca_scree_curve` builds data with six genuine signal dimensions (large variance) buried in nineteen noise dimensions, rotated so the signal is not aligned with any one measured axis, the setting PCA itself is built to unmix. The noise dimensions are not equal-variance: their variance decays geometrically, the shape a real Marchenko-Pastur noise spectrum actually takes, not the flat block a first pass at this example produced before that mismatch was caught. The resulting eigenvalue curve drops by an order of magnitude exactly at the signal/noise boundary, then keeps decaying, slowly and smoothly, rather than levelling into a plateau.

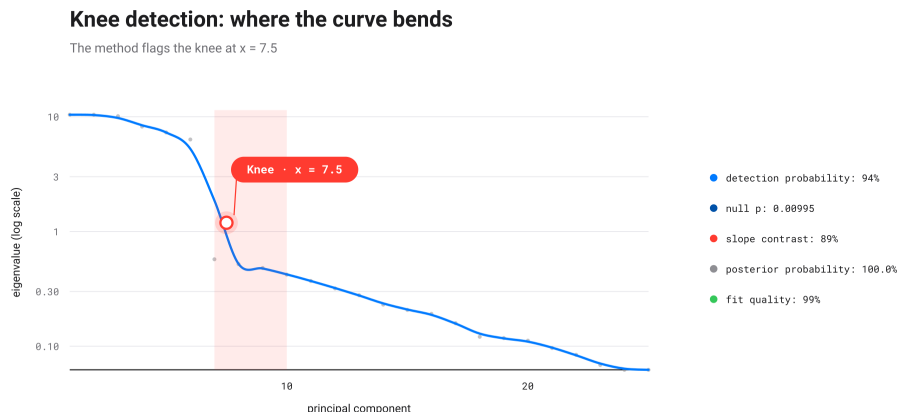


Figure 8: The PCA scree plot, with `elbow_helper.robust_elbow`’s detected elbow (dashed red line) at the boundary between 6 signal components and 19 noise components.

This example shares Section 6’s practical trap exactly: a scree plot is short and left-heavy in precisely the same way an inertia curve is, so it needs the same relaxed `RobustKneeConfig` profile. It also surfaces the convention worth knowing about before reading `knee_x` off any short applied curve: the locator’s “last point before the bend” rule consistently lands one to two components *past* the last true signal component, not exactly on it. Across every seed tested, the detected elbow falls at 6 or 7 components for a true signal dimension of 6, never below it, a systematic, predictable offset rather than noise.

On real data, without a known truth. The signal and noise above are built with a known signal dimension $d = 6$, set on purpose so a reader can check the answer. Real data never announces its own dimension. Take the same 178 wines and thirteen chemical measurements ([Aeberhard et al., 1992](#)) again, now read as thirteen continuous variables rather than a space to cluster: standardized, the covariance matrix computed, its eigenvalues sorted in decreasing order give a real scree plot,

with no signal-plus-noise construction underneath at all. `elbow_helper.robust_elbow`, called with the same short-curve profile, abstains again: `NoClearKnee`, reason `WEAK_SLOPE_CHANGE`. The curve decays, but without a single drop sharp enough for a slope change to clear the Theil-Sen contrast threshold of Section 15; thirteen chemical measurements that are realistically correlated with each other (alcohol and total phenols are not independent, for instance) spread variance across more components than an idealized signal-plus-noise construction ever would. A reader who had only seen this figure would have no way to guess a "correct" number of components to keep; that is precisely the kind of situation where an unqualified point answer would be most dangerous and where elbow-helper’s explicit abstention is worth the most.

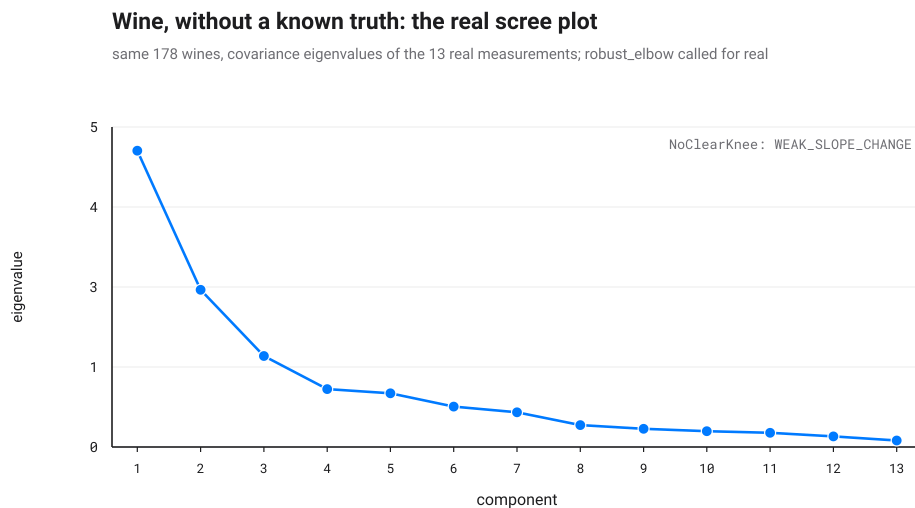


Figure 9: The real scree plot of the thirteen real chemical measurements of the same 178 wines (Aeberhard et al., 1992), with no known truth built underneath. `elbow_helper.robust_elbow` abstains (`NoClearKnee`, `WEAK_SLOPE_CHANGE`) rather than name an arbitrary component as the boundary.

27 A worked example: an exponential saturation curve

`examples/exponential_saturation.py` runs `robust_knee` against a shape none of the examples above have: $y = 1 - e^{-t/\tau}$, an RC-circuit charging voltage, a learning curve, a dose-response saturation, any quantity that rises fast then flattens toward a ceiling. Unlike a genuine broken-line knee, this curve has no true slope discontinuity anywhere: it is smooth and infinitely differentiable, a deliberate stress test for a locator built to find breaks in piecewise-linear trends.

The practical trap this example surfaces is a mismatch between two reasonable readings of “where the curve flattens”. The textbook time constant τ marks $1 - e^{-1} \approx 63\%$ of the total rise, a clean closed-form landmark. Across every seed tested, `robust_knee` instead lands consistently around $t/\tau \approx 1.9\text{--}2.0$, matching $1 - e^{-2} \approx 85\%$, the same “practically settled” convention used informally in RC-circuit and control-theory contexts. A human eye reads the curve the same way: at τ it is still visibly rising, not yet flat. This is not a bug to fix: it is what “last point before the bend” means for a curve that never truly stops bending: the located knee also depends on the observation window, here $t \in [0, 5\tau]$, since the decision is about maximum deviation from the chord spanning whatever

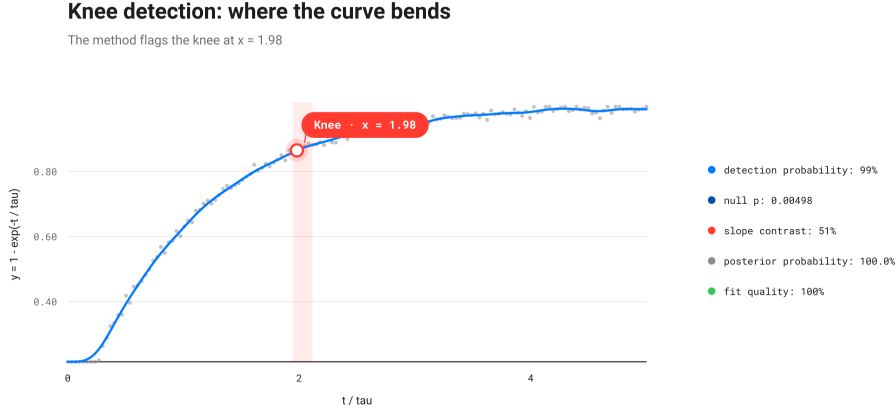


Figure 10: The RC-charging curve $y = 1 - e^{-t/\tau}$ with light noise, and `elbow_helper.robust_knee`'s detected knee (dashed red line and callout) at $t/\tau \approx 1.98$.

range the data actually covers, not a property of τ alone.

28 A worked example: queueing latency and a capacity-planning knee

`examples/queueing_latency.py` runs `robust_knee` on a shape none of the examples above take: a single-server queue's response time grows like $1/(1 - \rho)$ in its utilization ρ , a classic M/M/1-style blow-up. Latency stays close to the baseline service time for most of the range, then rises sharply as ρ approaches 1. This curve is convex and increasing rather than the concave, saturating shapes seen so far: the knee marks not a ceiling but an acceleration, the point past which a little more load costs a lot more latency.

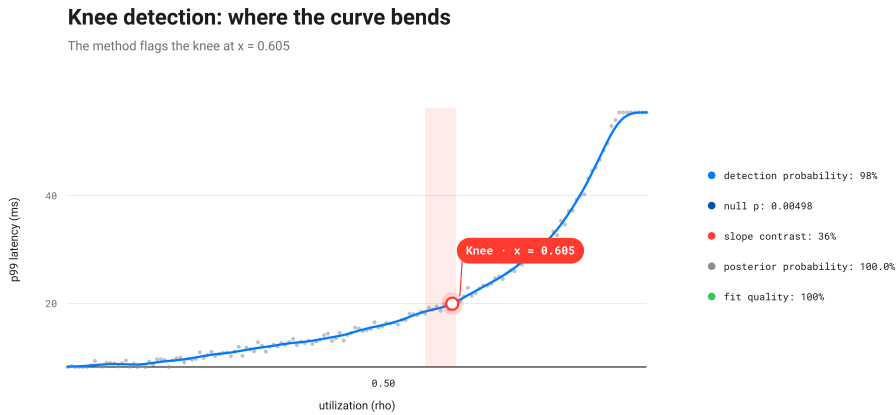


Figure 11: p99 latency versus utilization ρ , with `elbow_helper.robust_knee`'s detected knee (dashed red line and callout) at $\rho \approx 0.605$.

The knee lands at $\rho \approx 0.60$, well before the mathematical blow-up at $\rho = 1$, at a latency of

about $2.5\times$ the baseline service time. A queue is already paying a real tax at 60% utilization; the curve only looks flat up to there because $1/(1-\rho)$ is still small on an absolute scale. Sizing capacity off “does it still look roughly flat” rather than off this located knee is a common, avoidable way capacity plans run out of headroom earlier than expected.

29 A worked example: cache sizing and diminishing returns

`examples/cache_hit_rate.py` answers a familiar infrastructure question with the same tool: how big should a cache be? For a working set with roughly power-law-popular items, hit rate as a function of cache size C follows a Michaelis-Menten shape, $C/(C + K)$, with K the cache size at which half the working set is already resident: fast early gains, then a long, flattening tail. This is concave and increasing, the same broad family as the exponential saturation curve two sections back, but built from a rational function rather than an exponential and answering a cost question rather than a physical time-constant one.

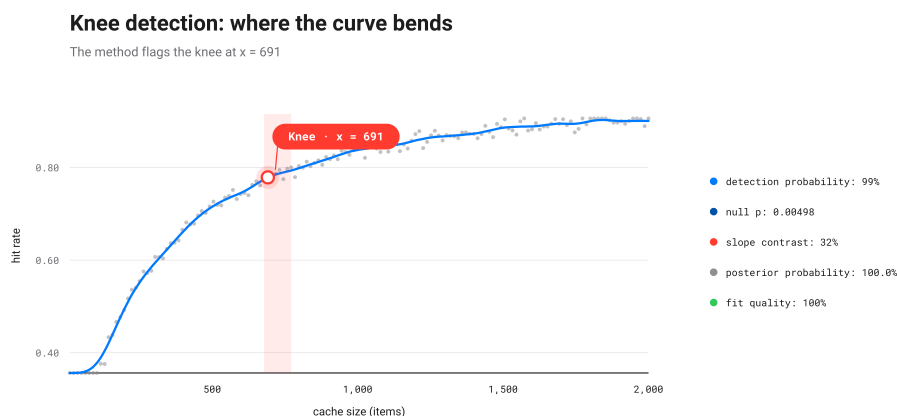


Figure 12: Hit rate versus cache size, with `elbow_helper.robust_knee`’s detected knee (dashed red line and callout) at 691 items, $3.46 \times K$.

The knee lands around $3.5K$, not at K itself. K only marks the 50%-hit-rate point, a property of the formula’s algebra, not of where the *marginal* return on cache size actually stops paying off. Buying cache up to K leaves a lot of cheap hit-rate on the table; buying past roughly $3.5K$ spends real memory for a shrinking sliver of hits, exactly the number a capacity-planning conversation needs and rarely has in closed form.

30 A worked example: when to stop testing

`examples/bug_discovery_rate.py` turns the same convex-decreasing family used by the k-means and PCA examples toward a release-engineering question instead of a clustering or dimensionality one. Reliability-growth models describe defect discovery during testing as fast at first (the easy, high-impact bugs) then slow (the rare, deep ones): a curve that falls steeply, bends, and flattens into a long, low tail.

The located knee sits about a day past the true break, the same small, systematic “one to two units past the true bend” offset the PCA scree example shows above. Reading `knee_x` as “stop

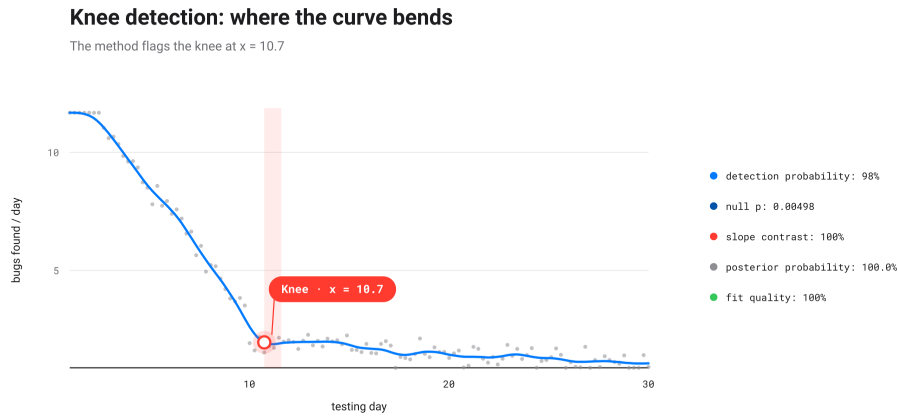


Figure 13: Bugs found per day of testing, with `elbow_helper.robust_knee`’s detected knee (dashed red line and callout) at day 10.7, one day past the true break at day 10.

testing exactly here” rather than “the discovery rate has now genuinely flattened, a day or two either side” over-trusts the point estimate past what it actually claims.

References

- Stefan Aeberhard, Danny Coomans, and Olivier de Vel. Comparison of classifiers in high dimensional settings. Technical Report 92-02, Dept. of Computer Science and Dept. of Mathematics and Statistics, James Cook University of North Queensland, 1992.
- Christophe Biernacki, Gael Celeux, and Gerard Govaert. Assessing a mixture model for clustering with the integrated completed likelihood. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(7):719–725, 2000.
- Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- Charles Bouveyron, Gilles Celeux, T. Brendan Murphy, and Adrian E. Raftery. *Model-Based Clustering and Classification for Data Science*. Cambridge University Press, 2021.
- Alice Cleynen, Thanh Mai Luong, Guillem Rigai, and Gregory Nuel. Fast estimation of the icl criterion for change-point detection problems, with applications to next-generation sequencing data. *Computational Statistics and Data Analysis*, 2013.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4th edition, 2022.
- Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani. *Algorithms*. McGraw-Hill, 2006.
- Robert B. Davies. Hypothesis testing when a nuisance parameter is present only under the alternative, 1977.
- Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.
- Klaus Frick, Axel Munk, and Hannes Sieling. Multiscale change point inference. *Journal of the Royal Statistical Society: Series B*, 2014.
- Piotr Fryzlewicz. Wild binary segmentation for multiple change-point detection. *The Annals of Statistics*, 42(6):2243–2281, 2014.
- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning*. Springer, 2nd edition, 2009.
- Robert E. Kass and Adrian E. Raftery. Bayes factors. *Journal of the American Statistical Association*, 90(430):773–795, 1995.
- Rebecca Killick, Paul Fearnhead, and Idris A. Eckley. Optimal detection of changepoints with a linear computational cost. *Journal of the American Statistical Association*, 107(500):1590–1598, 2012.
- Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.
- Guillem Rigai, Emilie Lebarbier, and Stephane Robin. Exact posterior distributions over the segmentation space and model selection for multiple change-point detection problems. *Statistics and Computing*, 22(4):917–929, 2012.

- Ville Satopaa, Jeannie Albrecht, David Irwin, and Barath Raghavan. Finding a kneedle in a haystack: Detecting knee points in system behavior. In *2011 31st International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2011.
- Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.
- Pranab Kumar Sen. Estimates of the regression coefficient based on kendall’s tau. *Journal of the American Statistical Association*, 63(324):1379–1389, 1968.
- Charles Spearman. The proof and measurement of association between two things. *American Journal of Psychology*, 15(1):72–101, 1904.
- Henri Theil. A rank-invariant method of linear and polynomial regression analysis. *Proceedings of the Royal Netherlands Academy of Sciences*, 53, 1950.
- Yi-Ching Yao. Estimating the number of change-points via schwarz’ criterion. *Statistics and Probability Letters*, 6(3):181–189, 1988.
- Nancy R. Zhang and David O. Siegmund. A modified bayes information criterion with applications to the analysis of comparative genomic hybridization data. *Biometrics*, 63(1):22–32, 2007.